

ROS 2

Arquitectura, Ecosistema y Flujo de Ejecución

Jesús Leonardo Cárdenas Martínez
Unidad Tamaulipas
Cinvestav
Ciudad Victoria, México
jesus.cardenas@cinvestav.mx

Abstract—Este documento describe la arquitectura de ejecución de ROS2 (Robot Operating System 2), siguiendo el flujo completo de información desde el comando escrito en la terminal hasta la instanciación de un módulo Python dentro de un nodo.

Se explican los mecanismos internos de cada capa: el entorno de ejecución del sistema operativo, el rol del ejecutable `ros2` como script Python, el sistema de `launch files` y sus mecanismos de sustitución diferida, el ciclo de vida de arranque de un nodo, el `Parameter Server`, y el `loop` de eventos en tiempo real basado en DDS.

El aporte de este documento es que un lector con conocimientos de Python y Linux pero sin experiencia previa en ROS 2 pueda comprender no solo qué ocurre, sino por qué ocurre, y sea capaz de trazar el recorrido de cualquier argumento desde la terminal hasta su uso en el código.

Index Terms—ROS 2, `launch files`, nodos, variables de entorno, DDS

I. INTRODUCCIÓN

Este documento es una guía de estudio sobre ROS 2 (Robot Operating System 2) orientada a explicar el *por qué* de cada decisión de diseño: por qué existe ROS 2 si ya existía ROS 1, por qué se eligió DDS como middleware, por qué el ecosistema está atado a versiones específicas de Ubuntu, y por qué distintas herramientas de simulación tienen distintos niveles de integración.

El documento está organizado de lo general a lo particular. Las primeras secciones cubren el contexto histórico: cómo nació ROS, qué limitaciones llevaron al rediseño, y cómo ese rediseño tomó forma en ROS 2. Las secciones siguientes bajan al nivel técnico del ecosistema: el middleware DDS, el esquema de distribuciones, los simuladores disponibles, y la arquitectura fundamental de nodos. El conjunto forma el mapa conceptual necesario para trabajar con ROS 2 de forma informada, no solo mecánica.

La motivación práctica de este documento es el trabajo de experimentación en control de formación multi-robot sobre ROS 2 Humble con Gazebo Classic 11. Comprender el ecosistema en el que se trabaja permite tomar decisiones de diseño fundamentadas y depurar problemas en el nivel correcto.

II. DE ROS A ROS 2

A. Origen de ROS

ROS (Robot Operating System) nació en 2007 en el laboratorio de inteligencia artificial de Stanford bajo el proyecto STAIR (Stanford Artificial Intelligence Robot). Su objetivo original era resolver un problema concreto: los equipos de robótica repetían los mismos componentes de software en cada proyecto nuevo porque no había una plataforma común sobre la que construir [1]. Willow Garage, una empresa de robótica fundada en 2006 en Menlo Park, adoptó el proyecto en 2008 y lo convirtió en el estándar de facto de la investigación robótica.

La propuesta de ROS era elegante: en lugar de definir una arquitectura monolítica, definió un conjunto de convenciones. Los procesos (nodos) se comunicaban mediante mensajes tipados sobre canales nombrados (tópicos), sin saber nada el uno del otro. El código podía reutilizarse entre robots completamente distintos porque la interfaz era siempre la misma: publicar y suscribirse a tópicos estándar. Un nodo de teleoperación escrito para un robot podía controlar cualquier otro robot que publicara en `/cmd_vel` con el tipo `geometry_msgs/Twist`.

El resultado fue una comunidad de contribución masiva. Para 2013 existían miles de paquetes ROS 1 que cubrían desde percepción visual hasta planificación de trayectorias. El robot PR2 de Willow Garage fue el catalizador: muchos laboratorios lo adoptaron y construyeron sobre el mismo stack de software, creando un ecosistema compartido sin precedente en robótica [1].

B. Limitaciones de ROS 1

El éxito de ROS 1 expuso sus limitaciones de diseño. ROS 1 fue construido con supuestos que funcionaban perfectamente en laboratorio pero que fallaban en producción [2]:

Punto único de fallo: el `rosmaster`. Toda comunicación en ROS 1 depende de un proceso central llamado `rosmaster`. Cuando un nodo quiere publicar en un tópico, registra su intención con el master. Cuando otro nodo quiere suscribirse, consulta al master por la dirección del publicador. El master es el directorio central del sistema. Si ese proceso falla, se desconecta, o la red entre él y algún

nodo se interrumpe, el sistema entero queda inutilizable. Esto hace imposible construir sistemas verdaderamente distribuidos en múltiples máquinas sin un punto central de fallo.

Sin garantías de comunicación. El protocolo de transporte de ROS 1 (TCPROS y UDPROS) no ofrece ninguna garantía de calidad de servicio. Los mensajes se entregan si el suscriptor está activo; si no, se pierden. No hay mecanismo de historial, persistencia, ni reintentos. Para un stream de cámara esto es aceptable, pero para comandos críticos es un riesgo real.

Python 2 y C++03. ROS 1 fue construido sobre Python 2, cuyo fin de vida oficial fue en enero de 2020. Mantener código ROS 1 después de esa fecha implica usar un intérprete sin soporte de seguridad. Las APIs de C++ también usaban estándares antiguos, limitando el uso de características modernas del lenguaje.

Sin soporte de tiempo real. El diseño de ROS 1 no contempló nunca requisitos de tiempo real. Los tiempos de entrega de mensajes no están garantizados, lo que impide su uso en lazos de control con restricciones temporales estrictas.

Sin seguridad. Toda la comunicación en ROS 1 es sin cifrado y sin autenticación. Cualquier nodo en la red puede suscribirse a cualquier tópico, inyectar mensajes, o incluso llamar a servicios. Esto es inaceptable en sistemas con acceso a internet o en entornos industriales.

C. El rediseño: ROS 2

ROS 2 no es una versión más de ROS 1. Es un rediseño completo que mantiene los conceptos de alto nivel (nodos, tópicos, servicios, parámetros) pero reemplaza toda la infraestructura subyacente [3]. El diseño fue guiado por cinco objetivos explícitos:

Eliminación del master. ROS 2 delega el descubrimiento de nodos al middleware DDS (Data Distribution Service), un estándar de la industria que implementa descubrimiento distribuido sin servidor central. Los nodos se anuncian entre sí directamente en la red usando el protocolo RTPS (Real-Time Publish Subscribe). La desaparición de un nodo no afecta a ningún otro.

Calidad de servicio configurable. Heredando las políticas QoS de DDS, cada publicador y suscriptor puede configurarse independientemente con distintos niveles de confiabilidad, historial y durabilidad. Esto permite que el mismo sistema use comunicación confiable para comandos de seguridad y comunicación de mejor esfuerzo para streams de alta frecuencia.

Soporte de tiempo real. La arquitectura de ROS 2 fue diseñada explícitamente para ser compatible con ejecutores de tiempo real. La capa de abstracción del middleware (RMW) puede ser reemplazada por implementaciones deterministas. La configuración completa requiere un kernel Linux con parches PREEMPT_RT y una implementación DDS que ofrezca garantías temporales.

Seguridad nativa. SROS 2 (Secure ROS 2) es la capa de seguridad integrada en ROS 2. Provee cifrado de comunicaciones, autenticación de nodos mediante certificados, y control de acceso por tópico. Se activa sin cambiar el código de aplicación.

Python 3 y C++ moderno. ROS 2 fue construido desde cero sobre Python 3 y C++14/17. Las APIs de `rclpy` y `rclcpp` usan características modernas: type hints, lambdas, smart pointers, y estructuras de datos estándar.

La arquitectura interna también cambió radicalmente. En ROS 1, la biblioteca de cliente era una sola capa monolítica. En ROS 2, la pila se divide en tres niveles: `rcl` (ROS Client Library), una biblioteca en C puro que implementa la lógica de ROS independiente del lenguaje; `rmw` (ROS Middleware), la capa de abstracción que oculta la implementación DDS concreta; y `rclpy/rclcpp`, los bindings de alto nivel para Python y C++ respectivamente. Esta separación permite cambiar la implementación DDS sin recompilar el código de la aplicación.

D. Coexistencia: el puente `ros1_bridge`

La transición de ROS 1 a ROS 2 no fue inmediata. Para facilitar la migración gradual, existe el paquete `ros1_bridge` [4], que actúa como traductor bidireccional entre los dos sistemas. El puente corre como un nodo simultáneamente registrado en ambos entornos: suscribe tópicos de ROS 1 y los republica en ROS 2, y viceversa.

Su uso requiere tener ambas distribuciones instaladas simultáneamente y sourcear ambos entornos en la misma terminal. Es una solución de migración, no una arquitectura de producción: la traducción introduce latencia adicional y no todos los tipos de mensajes tienen mapeo automático. Los proyectos nuevos no deben diseñarse con `ros1_bridge` como componente permanente.

III. SISTEMAS DISTRIBUIDOS

Un sistema distribuido es un conjunto de procesos independientes que cooperan para realizar una tarea común, comunicándose exclusivamente mediante intercambio de mensajes [5]. La palabra clave es *exclusivamente*: los procesos no comparten memoria, no comparten relojes, y no asumen nada sobre el estado interno del otro. Toda la coordinación ocurre a través de mensajes.

Un robot moderno encaja perfectamente en esta definición. Un proceso lee el LiDAR a 10 Hz, otro ejecuta el algoritmo de control a 20 Hz, otro publica odometría calculada desde encoders, otro recibe comandos de velocidad, otro visualiza el estado en RViz. Cada proceso puede estar en la misma computadora o en máquinas distintas conectadas por red. Cada uno tiene su propio ciclo de vida: puede arrancar, reiniciarse, o fallar sin que los demás se detengan.

A. El modelo publicador-suscriptor

El patrón de comunicación central en ROS 2 es el modelo publicador-suscriptor (pub-sub). Los procesos publican mensajes en canales nombrados llamados tópicos; otros

procesos se suscriben a esos tópicos para recibirlos. El publicador no sabe quién lo escucha: no conoce la dirección, el número, ni siquiera la existencia de los suscriptores. El suscriptor tampoco sabe quién publica.

Este desacoplamiento tiene consecuencias concretas. Agregar un nodo que registre todos los comandos de velocidad no requiere modificar ningún nodo existente: simplemente se suscribe a `/cmd_vel`. Reemplazar el nodo que calcula odometría por uno mejor no afecta a los consumidores del tópico: mientras el tipo de mensaje sea el mismo, la transición es transparente [1].

El desacoplamiento también facilita la depuración. Con herramientas como `ros2 topic echo` es posible interceptar cualquier tópico en tiempo real sin modificar ningún nodo del sistema. El sistema no sabe que alguien está escuchando.

B. Patrones adicionales de comunicación

Además del pub-sub, ROS 2 ofrece dos patrones complementarios:

Solicitud-respuesta (servicios). Cuando un proceso necesita una respuesta garantizada a una consulta, usa servicios. El cliente envía una solicitud y bloquea hasta recibir una respuesta del servidor. Es adecuado para operaciones cortas y discretas: activar un sensor, consultar el estado de un sistema, configurar un parámetro en tiempo de ejecución. La semántica es síncrona: el cliente espera.

Acciones. Para operaciones largas que requieren retroalimentación progresiva, ROS 2 provee acciones. El cliente envía un *goal* (objetivo) y recibe periódicamente mensajes de *feedback* mientras la operación progresa, hasta que llega el *result* final. El cliente puede cancelar el goal en cualquier momento. Es el patrón correcto para navegación autónoma, manipulación, o cualquier tarea cuyo tiempo de ejecución sea variable.

C. Serialización y fronteras de proceso

Una consecuencia fundamental de la distribución es que los datos deben cruzar fronteras de proceso. Dentro de un proceso, los objetos viven en memoria y pueden pasarse por referencia. Entre procesos distintos (o entre máquinas), eso es imposible: los datos deben convertirse a un formato transmisible (serialización), enviarse por la red o el bus del sistema, y reconstruirse al otro lado (deserialización).

En ROS 2, la serialización de mensajes sigue el formato definido por el estándar DDS-XTYPES. El usuario define los tipos de mensajes en archivos `.msg` con una sintaxis propia; la cadena de compilación de ROS 2 genera automáticamente el código Python y C++ de serialización correspondiente. Esta separación entre la definición del tipo y su implementación es lo que permite que nodos en Python y nodos en C++ se comuniquen sin problemas: ambos usan la misma representación en el cable.

D. Ausencia de reloj global

Una consecuencia menos obvia de la distribución es la ausencia de un reloj global compartido. Dos procesos en la misma máquina comparten el reloj del sistema operativo, pero dos procesos en máquinas distintas tienen relojes que pueden diferir por milisegundos o más. Cuando se fusionan datos de sensores de distintas fuentes, los timestamps deben ser consistentes para que la fusión tenga sentido físico.

En este trabajo el problema no surge de forma crítica, porque cada robot opera con su propio lazo de control periódico sin necesidad de fusionar flujos de sensores heterogéneos con timestamps de máquinas distintas. Sin embargo, tan pronto como se integren sensores adicionales o se requiera sincronización entre robots, ROS 2 provee herramientas específicas: el paquete `message_filters` ofrece políticas de sincronización por timestamp (`TimeSynchronizer`, `ApproximateTimeSynchronizer`) que permiten alinear mensajes de distintas fuentes antes de procesarlos [6].

IV. DDS: EL MIDDLEWARE DE COMUNICACIONES

DDS (Data Distribution Service) es un estándar de middleware publicado por el Object Management Group (OMG) en 2004 [7]. Define un modelo de comunicación pub-sub completamente distribuido: no hay servidor central, no hay broker, no hay proceso coordinador. Cada participante anuncia su presencia en la red y descubre a los demás de forma autónoma.

ROS 2 adoptó DDS como su capa de transporte en lugar de construir una solución propia. La razón fue pragmática: DDS ya resolvía los problemas que ROS 1 no podía resolver (descubrimiento distribuido, calidad de servicio, tiempo real, seguridad), y era un estándar de industria con múltiples implementaciones maduras probadas en sistemas críticos como aviónica y defensa [3].

A. Descubrimiento distribuido

El protocolo subyacente de DDS es RTPS (Real-Time Publish Subscribe Protocol). Cuando un nodo ROS 2 arranca, su implementación DDS inicia el proceso de descubrimiento en dos fases.

La primera fase es SPDP (Simple Participant Discovery Protocol): el participante difunde periódicamente un anuncio de su existencia por multicast UDP en la red local. Otros participantes reciben el anuncio y añaden al nuevo participante a su lista de participantes conocidos.

La segunda fase es SEDP (Simple Endpoint Discovery Protocol): una vez que dos participantes se conocen, intercambian información sobre sus endpoints (publicadores y suscriptores) mediante comunicación punto a punto. Si un publicador y un suscriptor comparten el mismo nombre de tópico y tipo de mensaje compatible, DDS establece el canal de comunicación automáticamente.

El resultado: los nodos ROS 2 se descubren y conectan entre sí sin ninguna configuración manual de IPs, puertos,

ni direcciones. Basta con que estén en la misma red y compartan el mismo `ROS_DOMAIN_ID`.

B. El `ROS_DOMAIN_ID`

El `ROS_DOMAIN_ID` es un entero entre 0 y 232 que delimita el espacio de descubrimiento. Solo los participantes con el mismo Domain ID se descubren entre sí. Esto permite correr múltiples sistemas ROS 2 independientes en la misma red física sin interferencia: un robot de prueba con `ROS_DOMAIN_ID=0` y un robot de producción con `ROS_DOMAIN_ID=1` son invisibles el uno para el otro aunque compartan la misma red WiFi.

En trabajo con múltiples computadoras (por ejemplo, robot + estación base), basta con que ambas tengan el mismo `ROS_DOMAIN_ID` exportado y estén en la misma subred. No se requiere configuración adicional.

C. Políticas QoS

Las políticas de calidad de servicio son la razón principal por la que DDS fue elegido sobre protocolos más simples. Cada publicador y suscriptor puede configurarse independientemente. Las más relevantes en la práctica son las siguientes.

Reliability. Define si la entrega de mensajes está garantizada. **RELIABLE** asegura que cada mensaje llegue al suscriptor, retransmitiéndolo si es necesario. **BEST_EFFORT** envía sin confirmación: si el mensaje se pierde en la red, se pierde. Para streams de sensores de alta frecuencia donde el dato más reciente es siempre más valioso que un dato atrasado, **BEST_EFFORT** es la elección correcta. Para comandos de seguridad o configuración, **RELIABLE** es obligatorio.

Durability. Define qué ocurre con los mensajes publicados antes de que el suscriptor exista. **VOLATILE** descarta todos los mensajes anteriores: el suscriptor solo recibe lo publicado después de suscribirse. **TRANSIENT_LOCAL** guarda los últimos N mensajes y los entrega al suscriptor cuando este aparece. Esto es útil para tópicos de configuración que se publican una sola vez al inicio del sistema.

History. Define cuántos mensajes pasados se conservan en cola cuando el suscriptor no los procesa suficientemente rápido. **KEEP_LAST(N)** conserva los últimos N mensajes. **KEEP_ALL** conserva todos, limitado por la memoria disponible.

Deadline. Permite especificar que un publicador debe publicar al menos cada cierto período. Si el publicador no cumple el deadline, el middleware notifica al suscriptor. Útil para detectar nodos que han dejado de funcionar.

Liveliness. Define cómo se declara y verifica que un publicador sigue activo. Útil para detectar fallos de nodos sin necesitar un mecanismo de heartbeat manual.

La Tabla I resume las políticas más usadas con sus valores típicos en sistemas robóticos.

TABLE I: Políticas QoS y sus valores típicos en robótica.

Política	Sensor / alta frec.	Comando / config.
Reliability	BEST_EFFORT	RELIABLE
Durability	VOLATILE	TRANSIENT_LOCAL
History	KEEP_LAST(1)	KEEP_LAST(10)

D. La capa RMW: abstracción de la implementación

DDS es un estándar, no una implementación. Existen múltiples implementaciones del estándar, cada una con distintas características de rendimiento, licencia y plataformas soportadas. Para que ROS 2 no quede atado a ninguna implementación específica, existe la capa RMW (ROS Middleware), que define una interfaz C abstracta que cualquier implementación DDS puede satisfacer [3].

Las implementaciones principales son las siguientes. **Fast DDS** (eProsima) es la implementación por defecto en ROS 2 Humble. Es de código abierto, tiene excelente soporte de la comunidad, y es la más documentada en el contexto de ROS 2 [8]. **Cyclone DDS** (Eclipse Foundation) tiene menor latencia en redes locales y es la alternativa más popular para sistemas donde la latencia es crítica [9]. **Zenoh** es un protocolo más moderno que extiende DDS con soporte nativo para comunicación a través de internet y dispositivos con recursos limitados; está soportado oficialmente desde ROS 2 Jazzy [10].

Cambiar la implementación no requiere recompilar el código de la aplicación. Basta con exportar la variable de entorno antes de lanzar el sistema, como muestra la Terminal 1:

Terminal 1: Selección de implementación DDS en tiempo de ejecución.

```
1 export RMW_IMPLEMENTATION=
   rmw_cyclonedds_cpp
2 ros2 launch simulador demo_pva.launch.py
```

Esta flexibilidad es uno de los puntos más fuertes de la arquitectura de ROS 2: el código de aplicación escrito con `rclpy` o `rclcpp` es completamente agnóstico respecto a qué implementación DDS está corriendo por debajo.

V. DISTRIBUCIONES Y COMPATIBILIDAD

A. El esquema de versiones

ROS 2 sigue un ciclo de lanzamiento sincronizado con Ubuntu LTS. El motivo es pragmático: Ubuntu LTS (Long Term Support) garantiza cinco años de soporte de seguridad para sus paquetes del sistema, y los paquetes de ROS 2 dependen fuertemente de las versiones del sistema: Python, CMake, OpenSSL, las bibliotecas de DDS, y los compiladores. Construir ROS 2 sobre una base de sistema estable reduce drásticamente los problemas de dependencias entre usuarios [11].

Las distribuciones LTS de ROS 2 se lanzan junto con las versiones LTS de Ubuntu (cada dos años, en abril) y reciben mantenimiento por cinco años. Las distribuciones no-LTS (a veces llamadas STD, Standard) se lanzan en los años intermedios, tienen soporte de un año, y sirven como terreno de prueba para características nuevas que eventualmente llegan a la siguiente LTS. Para proyectos de investigación de largo plazo, las distribuciones LTS son siempre la elección correcta.

El nombre de cada distribución sigue una convención de dos palabras aliteradas con nombres de animales en orden alfabético: Ardent, Bouncy, Crystal, Dashing, Eloquent, Foxy, Galactic, Humble, Iron, Jazzy. Cuando se nombra una distribución informalmente, basta con la primera palabra (“Humble”, “Jazzy”).

La Tabla II resume las distribuciones más relevantes con su plataforma base y estado de soporte.

TABLE II: Distribuciones de ROS 2 y compatibilidad con Ubuntu.

Distribución	Ubuntu	Tipo	Soporte
Foxy Fitzroy	20.04	LTS	Expirado (may. 2023)
Galactic Geochelone	20.04	STD	Expirado (dic. 2022)
Humble Hawksbill	22.04	LTS	Hasta may. 2027
Iron Irwini	22.04	STD	Expirado (nov. 2024)
Jazzy Jalisco	24.04	LTS	Hasta may. 2029

B. Qué cambia entre distribuciones

Cambiar de distribución de ROS 2 no es equivalente a actualizar un paquete Python con `pip`. Es un cambio de plataforma completo que puede afectar tres niveles distintos:

API de `rcpp` / `relcpp`. Las interfaces de programación evolucionan entre distribuciones. Algunas funciones se deprecian con advertencias en una versión y se eliminan en la siguiente. El sistema de parámetros, por ejemplo, sufrió cambios significativos entre Dashing y Foxy: el método para declarar parámetros y la semántica de valores por defecto cambió de forma incompatible hacia atrás. Migrar código entre distribuciones alejadas puede requerir ajustes no triviales, especialmente en la forma de configurar QoS, crear suscriptores con opciones avanzadas, o usar características de ciclo de vida de nodos.

Tipos de mensajes. Los paquetes de mensajes estándar (`std_msgs`, `geometry_msgs`, `sensor_msgs`, `nav_msgs`) son generalmente estables entre distribuciones. Los mensajes de paquetes de terceros pueden cambiar sin garantías de compatibilidad. El paquete Nav2 de navegación autónoma, por ejemplo, cambió la interfaz de sus acciones entre Humble y Jazzy [12], requiriendo adaptaciones en código que dependía de él.

Herramientas de construcción y sistema. `colcon` es el sistema de construcción estándar de ROS 2 y es compatible entre distribuciones. Sin embargo, la versión

de CMake, Python, y las herramientas de compilación de C++ cambian con Ubuntu. Código que compila sin problemas en Ubuntu 22.04 (Humble) puede fallar en Ubuntu 24.04 (Jazzy) por cambios en el compilador, cambios en las flags por defecto de seguridad, o cambios en cómo se resuelven dependencias de sistema.

C. Recomendación práctica

Para proyectos de investigación con horizonte de uno a tres años, la distribución LTS activa es siempre la elección más segura. Al momento de escribir este documento, **Humble Hawksbill** sobre Ubuntu 22.04 es la LTS con mayor ecosistema de paquetes de terceros disponibles, la mayor cantidad de documentación y ejemplos en internet, y soporte garantizado hasta mayo de 2027.

Jazzy sobre Ubuntu 24.04 es la LTS más reciente, pero aún tiene menos paquetes de terceros disponibles y algunos cambios de API (especialmente en Nav2 y en la interfaz de acciones) que requieren adaptación. Para proyectos que arrancan en 2025 o después, Jazzy es la elección natural.

Si el proyecto necesita correr en hardware embarcado (Raspberry Pi, NVIDIA Jetson, BeagleBone), la elección de distribución también está condicionada por qué versión de Ubuntu está disponible y optimizada para ese hardware. Las versiones de Jetpack de NVIDIA, por ejemplo, están atadas a versiones específicas de Ubuntu, lo que a veces fuerza a usar distribuciones de ROS 2 no-LTS o versiones antiguas, con el consiguiente compromiso en soporte a largo plazo.

La regla práctica: *no actualizar de distribución a mitad de un proyecto activo salvo que haya una razón técnica imperativa*. El costo de migración es significativo y el beneficio raramente justifica la interrupción.

VI. SIMULADORES Y COMPATIBILIDAD CON ROS 2

La simulación es un componente central en el desarrollo de sistemas robóticos. Permite probar algoritmos sin riesgo de dañar hardware, repetir experimentos en condiciones controladas, acelerar el tiempo de ejecución para pruebas de duración larga, y ejecutar múltiples instancias en paralelo para búsqueda de parámetros. En investigación de control de formación multi-robot, la simulación es prácticamente indispensable: orquestar cinco robots físicos simultáneamente para cada iteración de desarrollo sería inviable.

A. Gazebo Classic

Gazebo es el simulador más usado en el ecosistema ROS y su historia está directamente ligada a la de ROS 1. Fue creado por Andrew Howard y Nate Koenig en 2002 en la Universidad del Sur de California [13] y fue adoptado por Willow Garage como el simulador estándar de ROS 1. Durante más de una década fue la herramienta de facto para simulación robótica en investigación académica.

Arquitectura. Gazebo Classic opera con dos procesos separados: `gzserver` corre la simulación física y publica

el estado del mundo, mientras que `gzclient` es la interfaz gráfica que visualiza ese estado. Esta separación permite correr simulaciones sin GUI en servidores remotos o en pipelines de CI/CD. Los dos procesos se comunican mediante el protocolo interno de Gazebo (no DDS), lo que significa que `gzserver` puede correrse solo sin interfaz gráfica con el flag `-headless`.

Formato SDF. Los mundos y modelos en Gazebo se describen en SDF (Simulation Description Format), un formato XML propio de Gazebo. SDF define la geometría de colisión, la geometría visual, las propiedades de inercia, las articulaciones, y los plugins de sensores y actuadores. Es más expressive que URDF (Unified Robot Description Format) de ROS 1, aunque menos portable. Los archivos `.urdf` pueden convertirse a SDF automáticamente; los archivos SDF son el formato nativo de Gazebo.

Plugins. La integración entre Gazebo y ROS 2 se realiza mediante plugins del paquete `gazebo_ros_pkgs`. Un plugin es una biblioteca dinámica que Gazebo carga en tiempo de ejecución y que puede publicar o suscribirse a tópicos ROS 2. Los plugins más comunes son el plugin de diferencial (`gazebo_ros_diff_drive`), que simula la cinemática de un robot de tracción diferencial y publica odometría; el plugin de LiDAR (`gazebo_ros_ray_sensor`), que simula un escáner láser y publica mensajes `sensor_msgs/LaserScan`; y el plugin de cámara, que simula una cámara RGB y publica imágenes.

Física. Gazebo Classic soporta múltiples motores de física intercambiables: ODE (Open Dynamics Engine) es el motor por defecto y el más probado; Bullet, DART y Simbody son alternativas con distintas características de precisión y rendimiento. ODE es suficiente para la mayoría de aplicaciones de investigación en robots de ruedas.

B. New Gazebo (Gazebo Harmonic)

El nuevo Gazebo (anteriormente llamado Ignition Gazebo, ahora simplemente Gazebo) es una reescritura completa que comenzó en 2019 [13]. Su arquitectura es modular: en lugar de un simulador monolítico, es un conjunto de librerías independientes (`gz-sim`, `gz-physics`, `gz-rendering`, `gz-sensors`) que pueden usarse por separado. Esta modularidad facilita integrar motores de física alternativos (como Bullet o Drake) o renderers distintos (OGRE 2, Vulkan).

La integración con ROS 2 se realiza mediante el paquete `ros_gz_bridge`, que traduce mensajes entre el sistema de transporte interno de Gazebo (`gz-transport`, que usa `protobuf`) y los tópicos ROS 2. El bridge es configurable: se especifica qué tópicos de Gazebo se deben exponer a ROS 2 y con qué tipos de mensajes.

El nuevo Gazebo tiene mejor soporte de sensores (cámaras de profundidad, IMU, GPS, LIDAR 3D), mejor rendimiento gráfico, y una API de plugins más limpia. Sin embargo, su documentación aún está en desarrollo activo y la cantidad de ejemplos disponibles es significativamente menor que la de Gazebo Classic.

C. Tabla de compatibilidad

La Tabla III resume la compatibilidad oficial entre distribuciones de ROS 2 y versiones de Gazebo.

TABLE III: Compatibilidad entre ROS 2 y versiones de Gazebo.

ROS 2	Gazebo Classic	New Gazebo
Foxy	11	Citadel
Humble	11	Fortress / Harmonic
Jazzy	N/A	Harmonic (recomendado)

Gazebo Classic 11 sobre ROS 2 Humble es la combinación con mayor cantidad de documentación, tutoriales y paquetes de terceros disponibles. Es la elección pragmática para investigación en control de formación: el stack `gazebo_ros_pkgs` está muy maduro, los plugins de odometría y LiDAR funcionan de forma confiable, y la mayoría de robots de ejemplo disponibles en internet están descritos para esta combinación.

D. Alternativas

Webots. Webots es un simulador de código abierto desarrollado originalmente por la École Polytechnique Fédérale de Lausanne (EPFL) y actualmente mantenido por Cyberbotics [14]. Tiene integración oficial con ROS 2 mediante el paquete `webots_ros2`. Su interfaz gráfica es más amigable que la de Gazebo y su instalación es más simple (no depende de ROS). Es una buena alternativa para proyectos que priorizan facilidad de uso sobre profundidad de configuración.

NVIDIA Isaac Sim. Isaac Sim es el simulador de NVIDIA basado en la plataforma Omniverse [15]. Ofrece renderizado fotorrealista en tiempo real, simulación de sensores de alta fidelidad (LiDAR, cámara de profundidad, IMU), y capacidades de síntesis de datos para entrenamiento de redes neuronales. Requiere GPU NVIDIA con soporte RTX. Su integración con ROS 2 es oficial y bien documentada. Es la opción de mayor fidelidad disponible, pero también la de mayor costo computacional y complejidad de configuración.

Para el trabajo de control de formación multi-robot presentado en este documento, Gazebo Classic 11 sobre ROS 2 Humble es la elección que ofrece el mejor equilibrio entre madurez del ecosistema, disponibilidad de documentación, y simplicidad de configuración.

VII. ARQUITECTURA DE NODOS ROS 2

El nodo es la unidad básica de cómputo en ROS 2. Es un proceso con identidad propia en la red (un nombre único), uno o más canales de comunicación (tópicos, servicios, acciones), y un conjunto de parámetros configurables [16]. La imagen conceptual de un sistema ROS 2 en funcionamiento es un grafo: los nodos son los vértices y los tópicos son las aristas que los conectan. Herramientas como `rqt_graph` permiten visualizar este grafo en tiempo real.

Un nodo no es necesariamente un proceso del sistema operativo. ROS 2 soporta *composición*: varios nodos pueden ejecutarse dentro del mismo proceso para reducir la latencia de comunicación entre ellos (comunicación por memoria compartida en lugar de red). Sin embargo, en el modelo más común y en este trabajo, cada nodo es un proceso independiente.

A. Tópicos

Un tópico es un canal de comunicación asíncrono con nombre y tipo de mensaje fijo. Implementa el patrón pub-sub: cualquier nodo puede publicar en él y cualquier nodo puede suscribirse. El tipo de mensaje define la estructura de los datos que viajan por el canal y debe coincidir entre publicador y suscriptor. Si no coincide, DDS rechaza la conexión.

Los tipos de mensajes se definen en archivos `.msg` dentro de paquetes ROS 2. El tipo `geometry_msgs/msg/Twist`, por ejemplo, contiene dos vectores 3D que representan velocidad lineal y angular. El tipo `sensor_msgs/msg/LaserScan` contiene un array de distancias con metadatos de ángulo y frecuencia. La cadena de compilación de ROS 2 genera automáticamente las clases Python y C++ correspondientes.

Los nombres de tópicos son jerárquicos y pueden calificarse con un namespace. El tópico `cmd_vel` dentro del namespace `/robot0` se convierte en el tópico completo `/robot0/cmd_vel`. El namespace es la herramienta que permite que múltiples robots ejecuten el mismo código sin conflictos: cada uno opera dentro de su propio espacio de nombres.

La Terminal 2 muestra los comandos de inspección de tópicos más usados en la práctica:

Terminal 2: Herramientas de línea de comandos para inspección de tópicos.

```
1  ros2 topic list                # lista
   todos los topicos activos
2  ros2 topic echo /robot0/cmd_vel #
   imprime mensajes en tiempo real
3  ros2 topic hz /robot0/odom     # mide
   la frecuencia de publicacion
4  ros2 topic info /robot0/scan   #
   muestra tipo, publicadores y
   suscriptores
5  ros2 topic bw /robot0/scan     # mide
   el ancho de banda consumido
```

B. Servicios y acciones

Cuando el patrón pub-sub no es suficiente, ROS 2 ofrece dos patrones adicionales con semántica de respuesta.

Servicios. Un servicio implementa el patrón solicitud-respuesta síncrono. El cliente envía una solicitud con datos y bloquea hasta recibir la respuesta del servidor. Es apropiado para operaciones cortas y discretas: consultar el

estado de un componente, activar o desactivar un sensor, o configurar un parámetro en tiempo de ejecución. La limitación es que el cliente se bloquea durante la espera, lo que puede ser problemático en nodos con lazos de control de tiempo real. En esos casos se usan clientes asíncronos con futures.

Los tipos de servicio se definen en archivos `.srv` con dos secciones separadas por `--`: la definición de la solicitud y la definición de la respuesta.

Acciones. Las acciones resuelven el caso donde la operación es larga, su duración es variable, y el cliente necesita retroalimentación progresiva. El cliente envía un goal y recibe periódicamente mensajes de feedback hasta que llega el result final. El cliente puede cancelar el goal en cualquier momento. Es el patrón correcto para navegación autónoma, manipulación de objetos, o cualquier comportamiento cuyo tiempo de ejecución dependa del estado del mundo.

Los tipos de acción se definen en archivos `.action` con tres secciones: goal, result, y feedback.

La Terminal 3 muestra comandos de inspección de servicios desde la terminal:

Terminal 3: Inspección de servicios desde la terminal.

```
1  ros2 service list
2  ros2 service type /set_bool
3  ros2 service call /set_bool std_srvs/srv/
   SetBool "{data: true}"
```

C. Parámetros

Los parámetros son valores de configuración asociados a un nodo. Permiten modificar el comportamiento del nodo sin recompilarlo ni modificar su código. Cada nodo mantiene su propio Parameter Server: una base de datos en memoria que almacena los parámetros con su nombre, tipo, y valor actual.

Los parámetros tienen tipo estático: se declara el tipo al declararlos mediante el valor por defecto, y el Parameter Server solo acepta valores del mismo tipo. Los tipos soportados son `bool`, `int`, `double`, `string`, y arrays de cada uno de ellos.

Los parámetros pueden pasarse de tres formas. Desde la línea de comandos con el flag `-p`. Desde un archivo YAML con el flag `-params-file`. O modificarse en tiempo de ejecución mediante la CLI o desde otro nodo programáticamente. La Terminal 4 muestra los primeros dos casos:

Terminal 4: Paso de parámetros desde la línea de comandos y desde archivo YAML.

```
1 ros2 run simulador pva_node --ros-args -p
  vmax:=1.2
2 ros2 run simulador pva_node --ros-args --
  params-file config.yaml
```

El archivo YAML sigue una convención jerárquica fija donde el nombre del nodo es la clave raíz bajo `ros__parameters`:

Terminal 5: Estructura de un archivo de parámetros YAML.

```
1 pva_node:
2   ros__parameters:
3     vmax: 1.2
4     wmax: 1.0
5     n_rays_used: 5
6     d_safe: 0.3
```

En código Python, el flujo es siempre: primero declarar, luego leer. Llamar a `get_parameter` antes de `declare_parameter` produce un error:

Terminal 6: Declaración y lectura de parámetros en rclpy.

```
1 # Declarar (registra el nombre, tipo, y
  # valor por defecto)
2 self.declare_parameter('vmax', 0.5)
  # float
3 self.declare_parameter('n_rays_used', 0)
  # int
4
5 # Leer (consulta el Parameter Server, ya
  # resuelto)
6 vmax = self.get_parameter('vmax').
  value # float
7 n_rays_used = self.get_parameter('
  n_rays_used').value # int
```

D. Nodos de ciclo de vida

ROS 2 introduce el concepto de nodos de ciclo de vida (lifecycle nodes) como una extensión del nodo estándar. Un lifecycle node expone explícitamente una máquina de estados con transiciones bien definidas: *Unconfigured* → *Inactive* → *Active* → *Finalized*. Las transiciones son controladas desde fuera del nodo mediante un servicio especial.

Este modelo es útil en sistemas donde el orden de inicialización de los componentes importa: un nodo de control no debe activarse hasta que el nodo de percepción esté listo. Con lifecycle nodes esto es explícito y controlable,

en lugar de depender de tiempos de espera o verificaciones manuales.

E. Herramientas de línea de comandos

La CLI de ROS 2 es la principal herramienta de inspección y depuración de un sistema en tiempo de ejecución. La Terminal 7 reúne los subcomandos más utilizados:

Terminal 7: Comandos fundamentales de la CLI de ROS 2.

```
1 ros2 node list # nodos
  activos con su namespace
2 ros2 node info /robot0/consenso # topicos
  , servicios y parametros de un nodo
3 ros2 topic list # topicos
  activos
4 ros2 topic echo /robot0/odom # ver
  mensajes en tiempo real
5 ros2 topic hz /robot0/scan #
  frecuencia de publicacion
6 ros2 service list #
  servicios disponibles
7 ros2 param list /robot0/consenso
  # parametros del nodo
8 ros2 param get /robot0/consenso vmax
  # valor de un parametro
9 ros2 param set /robot0/consenso vmax 0.8
  # cambiar en tiempo real
10 ros2 run rqt_graph rqt_graph # grafo
  visual del sistema
```

El comando `ros2 param set` es especialmente útil durante la puesta a punto de algoritmos: permite ajustar ganancias o límites sin reiniciar el nodo, siempre que el nodo tenga registrado un callback de cambio de parámetro (`add_on_set_parameters_callback`).

VIII. EL ENTORNO DE EJECUCIÓN

Antes de que el sistema ROS 2 pueda ser invocado, el sistema operativo debe saber dónde encontrarlo. Esta sección explica la infraestructura sobre la que todo lo demás descansa.

A. Dónde se instala cada distribución

Como se describió en la sección anterior, cada distribución de ROS 2 ocupa su propio directorio dentro de `/opt/ros/`. Si coexistieran Humble y Jazzy instaladas simultáneamente, vivirían en `/opt/ros/humble/` y `/opt/ros/jazzy/` sin interferir entre sí. La distribución activa en una terminal es aquella cuyo `setup.bash` fue sourceado; la variable `ROS_DISTRO` confirma cuál es, como muestra la Terminal 8:

Terminal 8: Verificación de la distribución activa.

```
1 echo $ROS_DISTRO
2 # humble
```


B. Dónde vive ROS 2

ROS 2 Humble se instala en `/opt/ros/humble/`. El directorio `/opt/` es el lugar estándar en Linux para software opcional instalado en el sistema, es decir, software que no viene con el OS pero que se agrega encima. La Terminal 9 muestra la estructura interna, análoga a la del propio sistema operativo:

Terminal 9: Estructura de instalación de ROS 2 Humble.

```
1 /opt/ros/humble/
2 |-- bin/          <- ejecutables: ros2,
   rviz2, gazebo...
3 |-- lib/          <- librerías compiladas y
   paquetes Python
4 |-- include/      <- headers de C++
5 |-- share/        <- recursos y launch files
   del sistema
6 '-- setup.bash <- el script que se
   sourcea
```

El workspace del usuario replica exactamente esta estructura pero contiene únicamente los paquetes desarrollados por él, como se ve en la Terminal 10:

Terminal 10: Estructura del workspace tras compilar con `colcon build`.

```
1 ws3/install/
2 |-- control_nodes/
3 |   |-- bin/
4 |   |-- lib/python3.10/site-packages/
5 |-- simulador/
6 '-- setup.bash <- encadenador: sourcea
   ROS2 + workspace
```

C. Variables de entorno

Una variable de entorno es un par `NOMBRE=valor` almacenado en la memoria del proceso actual. No pertenece a ningún programa en particular, sino al entorno del proceso: la tabla de configuración global que el sistema operativo mantiene para cada proceso en ejecución.

`PATH` es la variable que contiene una lista de directorios separados por `:` donde el OS busca ejecutables. Si `/opt/ros/humble/bin` no está en `PATH`, el comando `ros2` devuelve `command not found`. `PYTHONPATH` cumple la misma función para módulos Python: sin esta variable configurada, la instrucción `import rclpy` falla con `ModuleNotFoundError`. `AMENT_PREFIX_PATH` es utilizada internamente por ROS 2 para localizar los paquetes instalados mediante `ament_index`. Finalmente, `ROS_DISTRO` contiene el nombre de la distribución activa y es leída por algunos paquetes para ajustar su comportamiento. Todas pueden inspeccionarse como indica la Terminal 11:

Terminal 11: Inspección de variables de entorno relevantes.

```
1 echo $PATH          # rutas donde el OS
   busca ejecutables
2 echo $PYTHONPATH    # rutas donde Python
   busca módulos
3 echo $ROS_DISTRO    # distribución de ROS2
   activa
4 env                 # muestra todas las
   variables
```

D. Qué es `source` y por qué es necesario

Cuando se ejecuta un script de shell de la manera convencional, el OS crea un proceso hijo, ejecuta el script en ese proceso hijo, y al terminar el proceso hijo desaparece llevándose consigo cualquier cambio a las variables de entorno. La terminal del usuario no es afectada. La Terminal 12 ilustra la diferencia:

Terminal 12: Diferencia entre ejecutar y sourcear un script.

```
1 # Ejecutar normal: modifica un proceso
   hijo que desaparece
2 bash setup.bash      # los cambios a PATH
   se pierden
3
4 # Sourcear: ejecuta las líneas
   directamente en la terminal
5 source setup.bash    # los cambios a PATH
   persisten
```

El comando `source` (equivalente al punto `.` en bash) ejecuta las líneas del script en el proceso actual de la terminal, como si se hubieran escrito directamente. Las modificaciones a `PATH` y `PYTHONPATH` quedan en ese proceso y son heredadas por todos los programas que se lancen desde ahí.

E. Qué hace `setup.bash` internamente

El archivo `ws3/install/setup.bash` generado por `colcon build` actúa como encadenador: primero sourcea ROS 2, luego el workspace del usuario. Su función central es invocar un script Python auxiliar que inspecciona los paquetes instalados en `ws3/install/` y genera dinámicamente los comandos `export` necesarios, como esquematiza la Terminal 13:

IX. EL EJECUTABLE ROS2

Terminal 13: Mecanismo central de `local_setup.bash` (simplificado).

```
1 # El script Python lee ws3/install/ y
  genera los export
2 _cmds="$(python3 local_setup_util_sh.py
  sh bash)"
3
4 # eval ejecuta esas lineas en la terminal
  actual
5 eval "${_cmds}"
6
7 # El resultado equivale aproximadamente a
  :
8 export PATH="/ws3/install/control_nodes/
  bin:$PATH"
9 export PYTHONPATH="/ws3/install/
  control_nodes/lib/\
10 python3.10/site-packages:$PYTHONPATH"
```

Esto es importante: `source setup.bash` no instala nada ni configura nada de forma permanente. Solo modifica las variables de entorno de la terminal actual. Al cerrar esa terminal, las modificaciones desaparecen. Por eso es necesario sourcear en cada terminal nueva.

F. El shebang y el PATH

El ejecutable `ros2` es un archivo de texto. La Terminal 14 muestra su primera línea:

Terminal 14: Primera línea del ejecutable `ros2`.

```
1 #!/usr/bin/env python3
```

Esta línea se llama shebang. Cuando Linux intenta ejecutar un archivo con permisos de ejecución, lee su primera línea: si comienza con `#!`, usa lo que sigue como intérprete y le pasa el archivo como argumento. `/usr/bin/env python3` no apunta a Python directamente; llama a `env`, que busca `python3` en el PATH actual. Como ya se sourceó ROS 2, ese `python3` tiene acceso a todos los módulos de ROS 2 a través de `PYTHONPATH`.

En consecuencia, escribir `ros2 launch ...` en la terminal es exactamente equivalente a lo que muestra la Terminal 15:

Terminal 15: Equivalencia entre el comando y su expansión real.

```
1 python3 /opt/ros/humble/bin/ros2 launch \
2     simulador demo_pva.launch.py \
3     n_rays_used:=5 vmax:=0.8
```

A. Cómo el OS entrega los argumentos

Cuando el sistema operativo ejecuta cualquier programa, le entrega todos los tokens escritos en la terminal como una lista de strings llamada `sys.argv`. Tomando como hilo conductor el comando de la Terminal 16, esa lista resultante es la que muestra la Terminal 17:

Terminal 16: Comando de ejemplo usado como hilo conductor.

```
1 ros2 launch simulador demo_pva.launch.py
  n_rays_used:=5 vmax:=0.8
```

Terminal 17: `sys.argv` recibido por el ejecutable `ros2`.

```
1 sys.argv = [
2     'ros2',
3     'launch',
4     'simulador',
5     'demo_pva.launch.py',
6     'n_rays_used:=5',
7     'vmax:=0.8'
8 ]
```

Esta es la frontera más baja del sistema. El OS no sabe si 5 es un entero, un float, o una cadena de texto. Todo es un string. Esta restricción se propaga hacia arriba y obliga a conversiones explícitas de tipo en cada capa.

B. Estructura modular de `ros2`

El ejecutable `ros2` es una CLI (Command Line Interface) modular. Cada subcomando (`launch`, `run`, `topic`, `node`, `param`) es un plugin Python separado que se carga bajo demanda. Al recibir `sys.argv[1] = 'launch'`, el ejecutable carga el plugin `ros2launch` y le delega el resto de la ejecución. Esto permite extender `ros2` con subcomandos personalizados sin modificar el ejecutable principal.

C. Parsing de los argumentos :=

El plugin `ros2launch` utiliza `argparse` de Python para separar el nombre del paquete y el archivo launch. Todo lo que queda después de esos dos campos es tratado como overrides de argumentos del launch file. La convención `clave:=valor` no pertenece a Python ni al OS; es una convención que ROS 2 definió y parsea manualmente mediante string splitting, tal como muestra la Terminal 18:

Terminal 18: Parsing de overrides de argumentos (simplificación del mecanismo interno).

```
1 launch_arguments = {}
2 for token in remaining_args:
3     if ':= ' in token:
4         key, value = token.split(':= ', 1)
5         launch_arguments[key] = value #
6             siempre string
7 # Resultado para el comando de la
8 # Terminal 1:
9 # launch_arguments = {'n_rays_used': '5',
10 # 'vmax': '0.8'}
```

El `split(':= ', 1)` asegura que si el valor contiene `:=` (por ejemplo, una ruta), solo se divide en la primera ocurrencia. El resultado es un diccionario de strings que se entrega al sistema de launch como condiciones iniciales.

D. Localización del launch file con `ament_index`

Una vez parseados los argumentos, `ros2launch` necesita localizar el archivo `demo_pva.launch.py` dentro del paquete `simulador`. Para ello usa `ament_index`: una base de datos de paquetes instalados que vive en `AMENT_PREFIX_PATH`. Cada paquete, al compilarse con `colcon build`, registra su nombre y ubicación en esa base de datos. La Terminal 19 representa lo que `ament_index` hace internamente:

Terminal 19: Localización del launch file (simplificado).

```
1 from ament_index_python.packages import \
2     get_package_share_directory
3
4 pkg_dir = get_package_share_directory('
5     simulador')
6 # -> '/ws3/install/simulador/share/
7     simulador'
8
9 launch_file = pkg_dir + '/launch/demo_pva
10     .launch.py'
```

Finalmente, `ros2launch` importa ese archivo como módulo Python normal, mediante `importlib.import_module`, y busca la función `generate_launch_description()`.

X. EL LAUNCH FILE

Un launch file es un script Python cuya única responsabilidad es describir qué nodos lanzar, con qué parámetros y en qué orden. `ros2 launch` lo importa como módulo y busca la función `generate_launch_description()`, que debe retornar un objeto `LaunchDescription`: una lista ordenada de acciones que el sistema procesa en secuencia.

Toda la configuración de parámetros pasa por el mismo par de piezas: `DeclareLaunchArgument` registra el argumento y establece su valor por defecto; `LaunchConfiguration` lo recupera después. El valor *siempre* viaja como string —ROS 2 nunca convierte tipos automáticamente.

Existen dos variantes del patrón según si se necesita o no ejecutar lógica Python sobre los valores.

A. Caso 1: parámetros escalares simples

Cuando los parámetros se pasan directamente al nodo sin transformación, `LaunchConfiguration` puede usarse como sustitución en línea. ROS 2 resuelve su valor en el momento de lanzar el proceso.

Terminal 20: Launch file sin lógica Python. `LaunchConfiguration` actúa como sustitución diferida.

```
1 from launch import LaunchDescription
2 from launch.actions import
3     DeclareLaunchArgument
4 from launch.substitutions import
5     LaunchConfiguration
6 from launch_ros.actions import Node
7
8 def generate_launch_description():
9     return LaunchDescription([
10         DeclareLaunchArgument('vmax',
11                               default_value='0.5'),
12         DeclareLaunchArgument('wmax',
13                               default_value='1.0'),
14
15         Node(
16             package='control_nodes',
17             executable='consenso_node',
18             namespace='robot0',
19             parameters=[{
20                 'vmax':
21                     LaunchConfiguration('
22                         vmax'),
23                 'wmax':
24                     LaunchConfiguration('
25                         wmax'),
26             }],
27         ),
28     ])
```

Este patrón es suficiente cuando no hay cálculo ni conversión de tipos: ROS 2 entrega el string al nodo y el nodo lo interpreta.

B. Caso 2: lógica Python sobre los argumentos

Cuando se necesita procesar el valor en Python —convertir tipos, iterar, calcular retardos— se añade `OpaqueFunction`. Al llegar a esa acción, el sistema llama a la función indicada y le pasa el *context*: el estado global de la ejecución que contiene todos los valores ya resueltos. Dentro de la función, `.perform(context)` extrae el string del context para que el código Python pueda operarlo.

XI. ARRANQUE DEL NODO

Terminal 21: Launch file con `OpaqueFunction`. Se usa cuando hay conversión de tipos o cálculos sobre los argumentos.

```
1 from launch import LaunchDescription
2 from launch.actions import
  DeclareLaunchArgument, OpaqueFunction
3 from launch.substitutions import
  LaunchConfiguration
4 from launch_ros.actions import Node
5
6 def launch_setup(context, *args, **kwargs):
7     vmax = float(LaunchConfiguration('
8         vmax').perform(context))
9     # Los waypoints llegan como string:
10    "1.0,0.0,4.0,0.0"
11    raw = LaunchConfiguration('waypoints
12    ').perform(context)
13    wps = [float(v) for v in raw.split(
14    ',')]
15
16    return [
17        Node(
18            package='control_nodes',
19            executable='
20                navigate_waypoints_pva',
21            namespace='robot0',
22            parameters=[{
23                'vmax':      vmax,
24                'waypoints': wps,
25            }],
26        )
27    ]
28
29 def generate_launch_description():
30     return LaunchDescription([
31         DeclareLaunchArgument('vmax',
32             default_value='0.5'),
33         DeclareLaunchArgument('waypoints',
34             default_value='
35                 4.0,0.0,4.0,4.0,0.0,4.0'),
36         OpaqueFunction(function=
37             launch_setup),
38     ])
```

La diferencia práctica: en el Caso 1, `LaunchConfiguration` es una promesa que ROS 2 resuelve solo; en el Caso 2, `.perform(context)` fuerza la resolución de inmediato para que el código Python pueda operar sobre el valor real.

C. Flujo de un valor de extremo a extremo

El tipo del argumento cambia solo donde el usuario escribe una conversión explícita: `int()` o `float()` en el launch file, e implícitamente al deserializar YAML en el Parameter Server. ROS 2 nunca convierte tipos por cuenta propia en ninguna de las etapas.

A. El mecanismo `entry_points`

Cuando el OS ejecuta `navigate_individual_pva`, no ejecuta directamente el archivo Python del usuario. Ejecuta un script wrapper¹ que `setuptools`² generó automáticamente durante `colcon build`, basado en la declaración de la Terminal 22.

Terminal 22: Declaración del ejecutable en `setup.py`.

```
1 entry_points={
2     'console_scripts': [
3         'navigate_individual_pva_ =
4         'control_nodes.
5             navigate_individual_pva:main'
6     ],
7 }
```

La sintaxis `nombre = modulo:funcion` le dice a `setuptools`: cuando alguien ejecute `navigate_individual_pva`, llama a la función `main()` del módulo `control_nodes.navigate_individual_pva`. La clave `console_scripts`³ es la categoría estándar para esto. Por ello, al agregar un nuevo ejecutable al paquete es necesario recompilar con `colcon build`: el wrapper no existe hasta que se genera.

B. La función `main()`

La Terminal 23 muestra la estructura de la función `main()`:

Terminal 23: Función `main()` típica de un nodo ROS 2.

```
1 def main(args=None):
2     rclpy.init(args=args)
3     node = NavigateIndividual()
4     rclpy.spin(node)
5     node.destroy_node()
6     rclpy.shutdown()
```

`rclpy.init(args=args)` es la primera llamada obligatoria. Inicializa la comunicación con el middleware DDS: establece la conexión con la red de nodos, configura el

¹Un wrapper (envolvente) es un script generado automáticamente que simplemente importa el módulo e invoca la función indicada. Actúa como puente entre el nombre del ejecutable en el sistema de archivos y la función Python real.

²`setuptools` es la biblioteca estándar de Python para empaquetar y distribuir código. ROS 2 la usa para compilar paquetes Python y generar sus ejecutables durante `colcon build`.

³`console_scripts` es la categoría estándar de Python para ejecutables de línea de comandos. Todo lo declarado aquí queda disponible como comando en la terminal después de instalar el paquete.

dominio (ROS_DOMAIN_ID), y prepara el Parameter Server⁴ del proceso. Sin esta llamada, ninguna funcionalidad de ROS 2 está disponible: ni suscriptores, ni publicadores, ni parámetros.

`rclpy.spin(node)` entrega el control al loop de eventos⁵ y bloquea el proceso ahí hasta que llegue una señal de terminación (Ctrl+C o SIGINT⁶). El código del usuario no vuelve a ejecutarse de manera lineal a partir de este punto; solo los callbacks⁷ y timers registrados en `__init__` vuelven a activarse.

C. El constructor `__init__`: secuencia y orden

El constructor del nodo es donde ocurre toda la configuración inicial. No es solo inicialización de variables: es el momento en que el nodo se anuncia en la red ROS 2, declara qué parámetros acepta, y establece todos sus canales de comunicación. El orden de las operaciones importa y no es intercambiable. La Terminal 24 muestra la secuencia completa:

⁴El Parameter Server en ROS 2 no es un proceso externo como en ROS 1. Es una base de datos en memoria interna a cada nodo que almacena sus parámetros con nombre, tipo y valor. Cada nodo tiene el suyo propio.

⁵Un loop de eventos (event loop) es un patrón de programación donde el proceso espera indefinidamente a que ocurran eventos externos (llegada de mensajes, temporizadores) y ejecuta funciones cortas (callbacks) en respuesta. El proceso nunca termina por sí solo; solo sale cuando recibe una señal explícita de terminación.

⁶SIGINT (Signal Interrupt) es la señal que el sistema operativo envía a un proceso cuando el usuario presiona Ctrl+C en la terminal. ROS 2 captura esta señal internamente y hace que `rclpy.ok()` retorne `False`, lo que provoca que `spin()` termine ordenadamente. Sin este manejo, el proceso se mataría abruptamente sin liberar recursos.

⁷Un callback es una función que no se llama directamente desde el código, sino que se registra para que el sistema la invoque automáticamente cuando ocurre un evento específico. Por ejemplo, `self.odom_callback` se registra para que ROS 2 la llame cada vez que llegue un mensaje de odometría, sin que el programador tenga que verificar manualmente la llegada de datos.

Terminal 24: Constructor completo de un nodo ROS 2 con parámetros, suscriptores y timer.

```
1 class NavigateIndividual(Node):
2     def __init__(self):
3         # 1. Registro en el grafo ROS2
4         super().__init__('navigate_individual')
5
6         # 2. Declaracion de parametros (
7             ANTES de leerlos)
8         self.declare_parameter('
9             n_rays_used', 0)
10        self.declare_parameter('vmax',
11                                1.0)
12
13        # 3. Lectura de parametros ya
14            resueltos
15        n_rays = self.get_parameter('
16            n_rays_used').value
17        vmax = self.get_parameter('vmax
18            ').value
19
20        # 4. Inicializacion de estado
21            interno
22        self.pose = None
23        self.odom = None
24
25        # 5. Suscriptores
26        self.create_subscription(Odometry
27                                , 'odom',
28                                self.
29                                    odom_cb
30                                    , 10)
31
32        # 6. Publicadores
33        self.cmd_pub = self.
34            create_publisher(
35                Twist, 'cmd_vel', 10)
36
37        # 7. Modulo de logica (Python
38            puro)
39        self.pva = PVA(n_rays_used=n_rays
40                        , v_max=vmax)
41
42        # 8. Timer de control (al final,
43            cuando todo esta listo)
44        self.create_timer(0.05, self.
45                            control_loop)
```

Por qué este orden. La llamada `super().__init__('navigate_individual')` debe ser la primera: registra el nodo en el grafo ROS⁸ con ese nombre y habilita todas las funciones de `self` (`declare_parameter`, `create_subscription`, `get_logger`, etc.). Antes de esa llamada, el objeto existe en Python pero no tiene ninguna capacidad de ROS 2.

⁸El grafo ROS 2 es la representación en tiempo real de todos los nodos activos, sus tópicos, servicios y acciones. Cuando un nodo se registra, los demás nodos pueden descubrirlo automáticamente a través de DDS sin configuración manual. Puede visualizarse con herramientas como `rqt_graph` o `rviz2`.

Los parámetros deben declararse antes de leerse. `get_parameter` solo funciona con parámetros previamente registrados mediante `declare_parameter`; llamarlo antes produce un error.

El timer se crea al final, una vez que todos los suscriptores, publicadores y módulos existen. Si el timer disparara antes de que `self.pva` estuviera construido, el callback intentaría usar un atributo que todavía no existe y Python lanzaría `AttributeError`⁹.

D. Estado interno y el problema del primer ciclo

En programación secuencial el código fluye de arriba hacia abajo: una instrucción se ejecuta después de la anterior. En un nodo ROS 2 no hay un flujo lineal después de `spin`: el programa se vuelve orientado a eventos¹⁰. El estado del nodo vive en variables de instancia (`self.pose`, `self.odom`) que los callbacks de suscripción actualizan cuando llegan mensajes, y que el timer lee cuando dispara.

El problema es que el timer puede disparar antes de que haya llegado el primer mensaje. Por eso `self.pose = None`¹¹: el valor `None` actúa como centinela que señala que aún no hay información disponible. El callback de control debe verificarlo antes de operar, como muestra la Terminal 25:

Terminal 25: Verificación de datos disponibles al inicio del loop de control.

```
1 def control_loop(self):
2     if self.pose is None or self.odom is None:
3         return # todavia no llego informacion, esperar
4
5     # a partir de aqui, self.pose y self.odom son validos
6     vel = self.pva.compute(self.pose, self.odom)
7     self.cmd_pub.publish(vel)
```

Sin esta verificación, el nodo fallaría en los primeros ciclos de control con un `AttributeError` o `TypeError`.

⁹`AttributeError` es la excepción de Python que se lanza al intentar acceder a un atributo de un objeto que no existe. Por ejemplo, `self.pva.compute()` fallaría con `AttributeError` si `self.pva` aún no fue asignado.

¹⁰La programación orientada a eventos es un paradigma donde el flujo del programa está determinado por eventos externos (mensajes, temporizadores) en lugar de por una secuencia predefinida de instrucciones. En lugar de preguntar “¿llegó un mensaje?” cada ciertos milisegundos (sondeo o polling), el nodo registra callbacks y el sistema lo despierta solo cuando ocurre algo relevante (notificación o event-driven). Esto ahorra CPU porque el proceso duerme mientras no hay eventos.

¹¹`None` es el valor especial de Python que representa ausencia de dato. Aquí se usa como valor centinela (sentinel): un marcador artificial que indica que una variable aún no ha recibido un valor real. Si todavía vale `None`, es porque el suscriptor no ha recibido ningún mensaje aún.

Es una fuente frecuente de bugs en nodos ROS 2 nuevos que parece funcionar en pruebas (porque los mensajes ya llegaron) y falla al arrancar en frío.

E. Declaración y lectura de parámetros en detalle

La Terminal 26 ilustra el mecanismo completo:

Terminal 26: Declaración y lectura de parámetros en el nodo.

```
1 self.declare_parameter('n_rays_used', 0)
2 self.declare_parameter('vmax', 1.0)
3
4 n_rays = self.get_parameter('n_rays_used').value # int 5
5 vmax = self.get_parameter('vmax').value # float 0.8
```

El segundo argumento de `declare_parameter` es el valor por defecto y también define el tipo esperado. Si se declara (`'vmax', 1.0`) (float) y desde la terminal se pasa `vmax:=texto`, ROS 2 lanzará un error de tipo. El tipo del default funciona como contrato¹²: el Parameter Server solo acepta valores del mismo tipo.

`get_parameter('vmax').value` accede al Parameter Server y retorna el valor en el tipo Python correspondiente: `int`, `float`, `str`, `bool` o `list`. Si desde la terminal llegó `vmax:=0.8`, el YAML lo deserializó a `float(0.8)`, y `.value` lo entrega ya convertido.

F. Las tres fronteras de serialización

El valor 5 escrito en la terminal recorre tres conversiones antes de estar disponible en Python. Cada conversión es un punto donde el tipo del dato puede cambiar o perderse, lo que explica muchos errores comunes en ROS 2:

TABLE IV: Fronteras de serialización del argumento `n_rays_used:=5`.

Frontera	De	A	Mecanismo
Terminal → launch	string '5'	string '5'	<code>split(':=')</code>
Launch → proceso	Python int 5	YAML 5	<code>-ros-args -p</code>
Proceso → nodo	YAML 5	Python int 5	<code>.value</code>

En la primera frontera no hay conversión: todo es string. La conversión a `int` ocurre en el launch file, donde el usuario la escribe explícitamente con `int(...)`. En la tercera frontera, el YAML se deserializa automáticamente y el tipo se infiere del valor declarado como default. Esto significa que el tipo final del parámetro depende de dos decisiones separadas: el `int()` en el launch file y el default en `declare_parameter()`. Si no coinciden, pueden ocurrir errores difíciles de depurar.

¹²Un contrato de tipo significa que una vez declarado un parámetro como float, el Parameter Server rechaza cualquier intento de asignarle un valor de otro tipo. Esto evita errores silenciosos donde un parámetro cambia de tipo inesperadamente.

XII. DEL NODO AL MÓDULO

A. Instanciación de módulos Python puros

Una vez recuperados los valores del Parameter Server, el nodo los usa para construir los objetos que implementan la lógica de control. En este punto el código abandona por completo el dominio de ROS 2. La Terminal 27 muestra cómo se instancia el módulo PVA:

Terminal 27: Instanciación del módulo PVA desde el nodo.

```
1 from control_nodes.algorithms.pva import
  PVA
2
3 self.pva = PVA(
4     n_rays_used=n_rays,      # int 5
5     v_max=vmax,              # float 0.8
6     d_safe=self.get_parameter('d_safe').
      value,
7     d_influence=self.get_parameter('
      d_influence').value,
8     xi=self.get_parameter('xi').value,
9     rp=0.25,
10 )
```

`n_rays` es un entero Python normal. La clase PVA no sabe nada de ROS 2, no sabe que existe un launch file, no sabe que hubo una terminal. Recibe un argumento de constructor como cualquier clase Python.

B. La clase PVA como Python puro

La Terminal 28 muestra el constructor de la clase PVA, que es Python estándar sin ninguna dependencia de ROS 2:

Terminal 28: Constructor de la clase PVA.

```
1 class PVA:
2     def __init__(self,
3         d_safe=0.3,
4         d_influence=1.0,
5         xi=1.0,
6         rp=0.25,
7         v_max=0.5,
8         w_max=0.5,
9         n_rays_used=None):
10
11         self.d_safe      = d_safe
12         self.d_influence = d_influence
13         self.xi          = xi
14         self.rp          = rp
15         self.v_max       = v_max
16         self.w_max       = w_max
17         self.n_rays_used = n_rays_used
```

Desde aquí en adelante es orientación a objetos Python estándar. La separación entre el código de ROS 2 (el nodo) y la lógica de la aplicación (el módulo) es una práctica

arquitectónica deliberada: permite probar la clase PVA de forma independiente, sin necesitar un contexto ROS 2 activo.

C. Por qué esta separación es importante

El nodo actúa como adaptador: su responsabilidad es recibir datos de la infraestructura ROS 2 (tópicos, parámetros) y entregarlos a los módulos de lógica de control en los tipos correctos. Los módulos implementan los algoritmos sin acoplarse al middleware. Esto tiene consecuencias concretas. En cuanto a testabilidad, los módulos pueden probarse con `unittest` sin inicializar `rclpy`. En cuanto a portabilidad, el mismo módulo puede usarse en otro sistema de middleware cambiando solo el nodo adaptador. En cuanto a claridad, la complejidad de ROS 2 queda confinada al nodo y la lógica de control es matemáticamente pura.

XIII. EJECUCIÓN EN TIEMPO REAL

A. El loop de eventos: `rclpy.spin()`

Una vez que el nodo está construido y todos sus suscriptores, publicadores y timers registrados, la llamada `rclpy.spin(node)` entrega el control al loop de eventos de ROS 2. La Terminal 29 muestra su comportamiento conceptual:

Terminal 29: Comportamiento conceptual de `rclpy.spin()`.

```
1 # Pseudocódigo del loop de eventos
2 while rclpy.ok():
3     esperar_evento()          # bloquea
4     ejecutar_callback()      # procesa
5     # repetir
```

El proceso permanece bloqueado en este loop indefinidamente. El código del usuario solo vuelve a ejecutarse cuando algún evento lo activa: la llegada de un mensaje en un tópico suscrito, o el disparo de un timer. Fuera de esos momentos, el proceso duerme y no consume CPU.

B. Suscripciones y callbacks de mensajes

Una suscripción registra un callback que ROS 2 llama automáticamente cada vez que llega un mensaje en el tópico indicado, como muestra la Terminal 30:

Terminal 30: Registro de una suscripción a odometría.

```
1 self.odom_sub = self.create_subscription(  
2     Odometry,          # tipo del  
3     'odom',           # nombre del  
4     self.odom_callback, # funcion a  
5     10                # queue size (QoS  
6     History)
```

El argumento 10 es el tamaño de la cola: si los mensajes llegan más rápido de lo que el callback los procesa, se acumulan hasta 10 mensajes antes de empezar a descartarlos. DDS gestiona la entrega desde el publicador (el nodo de odometría de Gazebo) hasta este callback, incluso si están en procesos diferentes.

El callback de suscripción típicamente solo actualiza el estado interno del nodo:

Terminal 31: Callback de suscripción: actualiza estado, no computa control.

```
1 def odom_callback(self, msg: Odometry):  
2     self.odom = msg # guarda el mensaje  
3     # NO computar control aqui: llega a  
4     # frecuencia variable
```

La razón de no computar el control dentro del callback de suscripción es que los mensajes de odometría llegan a la frecuencia que el publicador decida (o que la red permita), no a la frecuencia que el algoritmo de control necesita. Mezclar la lógica de control con la llegada de datos produce un nodo con frecuencia de operación impredecible.

C. Timers: el ciclo de control

Un timer registra un callback que se ejecuta periódicamente con frecuencia determinista. La Terminal 32 muestra cómo crear el loop de control a 20 Hz:

Terminal 32: Registro de un timer de control a 20 Hz.

```
1 self.timer = self.create_timer(  
2     0.05,              # periodo en  
3     segundos (20 Hz)  
4     self.control_loop # funcion a  
5     llamar
```

El timer garantiza que `control_loop` se ejecute cada 50 ms independientemente de cuándo llegaron los últimos

mensajes. Esta separación entre la *adquisición de datos* (callbacks de suscripción) y el *cómputo de control* (callback del timer) es el patrón fundamental en nodos de control con ROS 2.

D. El patrón completo: adquisición y control separados

La Terminal 33 muestra el patrón completo de un nodo de control tal como se estructura en este proyecto:

Terminal 33: Patrón completo de nodo de control: suscriptores actualizan estado, timer computa y publica.

```
1 def odom_callback(self, msg):  
2     self.odom = msg # actualiza  
3     estado: 50-100 Hz  
4 def scan_callback(self, msg):  
5     self.scan = msg # actualiza  
6     estado: 10 Hz  
7 def control_loop(self): # ejecuta a  
8     20 Hz exactos  
9     # guardia: datos aun no disponibles  
10    if self.odom is None or self.scan is  
11    None:  
12    return  
13  
14    # computo de control con datos mas  
15    recientes  
16    cmd = self.pva.compute(self.odom,  
17    self.scan)  
18  
19    # publicacion del comando de  
20    velocidad  
21    self.cmd_pub.publish(cmd)
```

Este patrón resuelve el problema de la asincronía inherente a los sistemas distribuidos: el nodo no bloquea esperando datos, sino que usa los datos más recientes disponibles en el momento en que el timer dispara. Si el LiDAR llega a 10 Hz y el control corre a 20 Hz, cada par de ciclos de control usará el mismo scan, lo cual es completamente aceptable.

E. Modelo de concurrencia: *SingleThreadedExecutor*

Por defecto, `rclpy.spin()` utiliza un *SingleThreadedExecutor*, lo que significa que todos los callbacks (suscriptores y timers) se ejecutan en el mismo hilo, uno a la vez. Esto tiene dos consecuencias:

Primero, el acceso a variables compartidas como `self.odom` y `self.scan` es seguro sin necesidad de locks: el executor garantiza que nunca habrá dos callbacks ejecutándose simultáneamente. Segundo, si un callback tarda más que su periodo (por ejemplo, el loop de control tarda 60 ms siendo el periodo 50 ms), el siguiente disparo del timer se retrasa. Los callbacks deben ser lo suficientemente rápidos para no acumular retraso.

Si se necesita paralelismo real (procesamiento de imagen pesado mientras el control sigue corriendo), existe `MultiThreadedExecutor` con `ReentrantCallbackGroup`, pero requiere manejo explícito de acceso concurrente a las variables compartidas.

F. El rol de DDS en la entrega de mensajes

DDS es la capa de transporte subyacente de ROS 2. Cuando un nodo arranca, DDS anuncia su existencia en la red y otros nodos que suscriben al mismo tópico se conectan automáticamente, sin configuración manual de IPs o puertos. Para la transmisión, DDS serializa los mensajes al formato wire, los transmite (localmente vía memoria compartida, o en red vía UDP) y los deserializa en el receptor.

El usuario de ROS 2 no interactúa con DDS directamente; `rclpy` lo abstrae completamente. Sin embargo, entender que DDS existe explica por qué los nodos se descubren automáticamente y por qué es posible comunicar nodos en máquinas diferentes con solo configurar `ROS_DOMAIN_ID`.

XIV. RESUMEN DEL FLUJO COMPLETO

La Tabla V condensa el recorrido completo del argumento `n_rays_used:=5` a través de las cuatro capas del sistema.

TABLE V: Flujo completo del argumento `n_rays_used:=5` desde la terminal hasta el módulo.

Capa	Forma del valor	Mecanismo	Responsable
Terminal	string '5'	<code>sys.argv</code> / <code>split(':=')</code>	OS / ros2
Launch file	string '5' → int 5	<code>.perform(ctx)</code> + <code>int(...)</code>	Usuario
Proceso	YAML <code>n_rays: 5</code>	<code>-ros-args -p</code>	ROS 2
Nodo	int 5	<code>get_parameter.value</code>	<code>rclpy</code>
Módulo	int 5	argumento constructor	Python

El diagrama siguiente ilustra las entidades involucradas en cada nivel y los mecanismos de cruce entre ellos:

Terminal 34: Diagrama de flujo Terminal → Módulo.

1	TERMINAL	
2	"n_rays_used:=5_vmax:=0.8"	
3	sys.argv + split(':=')	
4	v	
5	LAUNCH FILE [generate_launch_description]	
6	DeclareLaunchArgument	<- registra y almacena
7	OpaqueFunction Python	<- logica
8	.perform(context)	<- resuelve el string
9	int('5') = 5	<- conversion: usuario
10	Node(parameters={...})	<- descriptor (no proceso)
11	serializa a YAML, pasa como --ros-args -p	
12	v	
13	PROCESO [navigate_individual_pva --ros-args ...]	
14	rclpy.init()	<- inicia DDS
15	Node.__init__()	<- registra en el grafo
16	declare_parameter()	<- almacena en param server
17	get_parameter().value	<- recupera como int
18	argumento de constructor Python normal	
19	v	
20	MODULO [class PVA]	
21	self.n_rays_used = 5	<- Python puro

A. Observaciones finales

Tres conclusiones emergen del análisis de este flujo.

Todo empieza como string. La restricción del OS de que `sys.argv` es siempre una lista de strings se propaga hacia arriba. ROS 2 nunca convierte tipos automáticamente. Esa responsabilidad recae en el usuario dentro del launch file.

Cada capa tiene su propio mecanismo de serialización. El cruce entre capas requiere una conversión de formato: string splitting en la terminal, YAML al pasar al proceso, deserialización en el Parameter Server. Comprender estos mecanismos permite depurar errores de tipo que de otro modo son opacos.

El módulo de lógica no sabe que existe ROS 2. La arquitectura de separación nodo-módulo no es accidental: confina la complejidad del middleware a una capa de adaptación y deja el código de control matemáticamente puro, testeable y portable.

REFERENCES

- [1] M. Quigley, K. Conley, B. Gerkey, J. Faust, T. Foote, J. Leibs, R. Wheeler, and A. Y. Ng, “ROS: an open-source robot operating system,” in *ICRA Workshop on Open Source Software*, vol. 3, no. 3.2, 2009, p. 5.
- [2] Y. Maruyama, S. Kato, and T. Azumi, “Exploring the performance of ROS2,” in *Proceedings of the 13th International Conference on Embedded Software (EMSOFT)*. ACM, 2016, pp. 1–10.
- [3] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, “Robot operating system 2: Design, architecture, and uses in the wild,” *Science Robotics*, vol. 7, no. 66, p. eabm6074, 2022.
- [4] Open Robotics. (2022) ros1_bridge: A bridge between ROS 1 and ROS 2. [Online]. Available: https://github.com/ros2/ros1_bridge
- [5] A. S. Tanenbaum and M. Van Steen, *Distributed Systems: Principles and Paradigms*, 2nd ed. Pearson Prentice Hall, 2007.
- [6] Open Robotics. (2022) ROS 2 api reference: rclpy and message_filters. [Online]. Available: <https://docs.ros.org/en/humble/Tutorials.html>
- [7] Object Management Group, “Data Distribution Service (DDS) specification, version 1.4,” Object Management Group (OMG), Specification formal/2015-04-10, 2015.
- [8] eProsima. (2023) Fast DDS documentation. [Online]. Available: <https://fast-dds.docs.eprosima.com/>
- [9] Eclipse Foundation. (2023) Eclipse Cyclone DDS documentation and source. [Online]. Available: <https://github.com/eclipse-cyclonedds/cyclonedds>
- [10] ZettaScale Technology. (2023) Zenoh: Zero overhead publish/subscribe, store/query and compute. [Online]. Available: <https://zenoh.io/docs/>
- [11] Open Robotics. (2024) ROS 2 distributions and releases. [Online]. Available: <https://docs.ros.org/en/rolling/Releases.html>
- [12] S. Macenski, F. Martin, R. White, and J. G. Clavero, “The marathon 2: A navigation system,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2020, pp. 2718–2725.
- [13] N. Koenig and A. Howard, “Design and use paradigms for Gazebo, an open-source multi-robot simulator,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, vol. 3. IEEE, 2004, pp. 2149–2154.
- [14] Cyberbotics Ltd. (2019) Webots: Open-source robot simulator. [Online]. Available: <https://cyberbotics.com>
- [15] NVIDIA Corporation. (2023) NVIDIA Isaac Sim documentation. [Online]. Available: <https://docs.omniverse.nvidia.com/isaacsim/latest/overview.html>
- [16] Open Robotics. (2022) ROS 2 humble hawkbill documentation. [Online]. Available: <https://docs.ros.org/en/humble/>